How does a Proportional-Integral-Differential (PID) control system contribute to precise movement in robotics

The basics of PID controllers used in VEX robotics

Cheyns Ruben

6c - Wetenschappen-Wiskunde[2]

Scheppersinstituut Mechelen

2025-2026

Leerkrachten: Van Nuffel Joke, Vanhaecht Lies

Abstract

In this research paper the use of PID controllers in robotics will be discussed. Starting by mathematically explaining PID machine control and the different PID terms. This paper also contains information about PID tuning and can be both used as source or as tuning guide. Further, PID controllers implemented into VEX robotics get discussed using a python example.

Contents

1 INTRODUCTION

1.1 What is a Proportional-Integral-Derivative controller?

When discussing a Proportional-Integral-Derivative (PID) controller, a form of machine control is meant. Machine control refers to the process of operating a plant to reach a desired goal. This "plant" in machine control refers to the system being controlled. Machine control can be as simple as making a motor spin or as complex as completing a spacecraft journey. In this paper, the focus will be on simple machine control systems using only one actuator (electronic part performing an action) and sensor with the basic structure shown in the following figure:

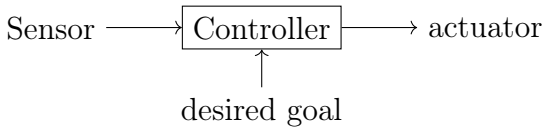

Figure 1: Basic structure of a machine control system. (own image)

To understand what a PID controller is, its use case and structure must be analyzed. The PID system is a feedback system, this means that the controller will use a sensor as input to compute a correcting signal that controls an actuator such as a motor. Most often this system is implemented in software using a (micro)computer connecting both the sensor and the actuator.

The controller (u) computes a correcting signal by combining three different terms, each multiplied by a respective constant (K) as shown in the following equation:

$$u(t) = K_P \cdot P(t) + K_I \cdot I(t) + K_D \cdot D(t)$$

The three terms are the proportional change (P), integral (I) and derivative (D) of a calculated error (e). This error (e) is calculated by subtracting a given value the plant is supposed to reach with the current measured value.

$$e(t) = sensorValue(t) - requestedSetpoint$$

The constants K_P K_I and K_D are different in each plant and are acquired through PID tuning, which will be discussed in the next chapter.

Consider this simple real life example: a radiator heats a room to a desired warmth.

To control the power of the radiator, a form of machine control is required. The radiator can namely not be set to a specific temperature but only to a certain power level. A PID controller can be installed in the thermostat regulating the radiator (actuator) using inputs from a thermometer (sensor).

By calculating the error using the requested temperature and current temperature ac-

quired via the thermometer, the three control terms can be calculated and combined to acquire a signal controlling the radiator. (VanDoren, 2003)

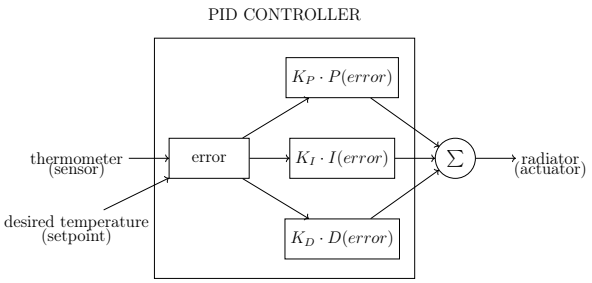

Figure 2: The structure of a PID controlled thermostat system.(own image)

1.1.1 Proportional term

The proportional term $P(t)$ within a PID system refers to the difference between the current value and the requested setpoint.

$$P(t) = e(t)$$

The proportional term is relatively simple to implement since it requires minimal calculations and is not prone to fluctuations in the input signal since it only uses the current error. Its main drawback is a possible steady-state-error, this means that a controller using only the proportional term will generally not reach the setpoint but only come close.

(Sufyan, 2025) (Shoujun & Weiguo, 2011)

When applying only this proportional factor to our thermostat example, the difference between the requested and current temperature gets calculated. Then the radiator is set to this value multiplied by a constant. This will result in heating more quickly initially, but gradually slowing down when nearing the requested temperature as shown in the following graph.

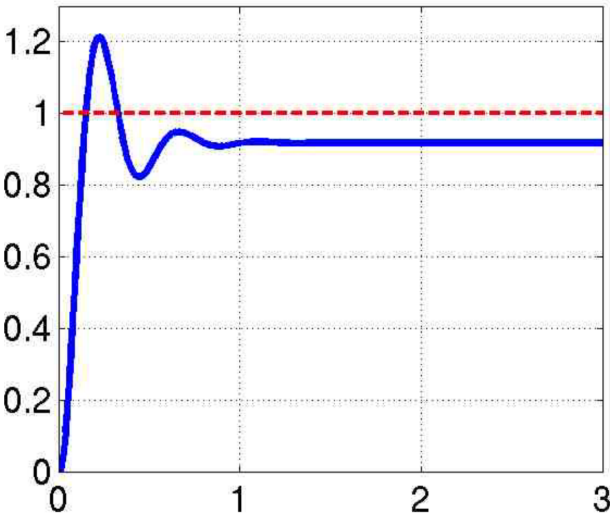

Figure 3: Graph showing the response of a proportional-only controller (Chen, 2011)

1.1.2 Integral term

The integral term $I(t)$ within a PID system is the integral of the error values over time. The term accumulates over past errors.

$$I(t) = \int e(t)dt$$

This accumulation makes the integral term effective to fight steady-state-error when not reaching the setpoint. This is because the integral term will keep increasing as long as the setpoint has not been reached, causing the outputted control signal to increase until the setpoint is finally reached.

However, the integral term can be prone to

windup since it will keep increasing when not having reached the setpoint and only decreases if the setpoint has been reached. That is why the integral term often causes heavy overshoots or oscillations. Oscillation is the repeated fluctuation around the setpoint.

The term can even cause the output to become saturated. This takes place when the outputted signal is higher than the physical maximum the actuator can achieve. Therefore, the maximal possible output is reached. This means that the integral value can keep increasing when the actuator is at its maximal output, causing a heavy overshoot when the error finally decreases.

(Sufyan, 2025) (Shoujun & Weiguo, 2011)

To decrease this possible overshoot, oscillation and saturation, multiple anti-windup methods can be implemented into a PID system. One of these methods is clamping, where the integral value is simply set to maximally be equal to the maximal value of the actuator. Using clamping, the integral factor will for example be set to maximally 60 if the controller drives a motor with a maximal speed of 60 RPM.

Other methods than clamping include disabling the integral term when the error is too high or adjusting it based on the controller output.

When applying only the integral term to the thermostat example, we will combine all previous errors and multiply them by the time between our sensor readouts, also called the sampling time, to get the integral value. This integral value will then be multiplied by a constant to get the control signal for the radiator.

The controller output will result in the radiator gradually building up in power and start to diminish in power only when the current temperature is higher than the requested temperature. The temperature will often overshoot and fluctuate around the requested temperature before stabilizing. An integral-only controller is therefore not recommended in temperature control. Most often the integral term will be combined with the proportional term to get a PI controller, capable of reaching the requested value with minimal overshoot.

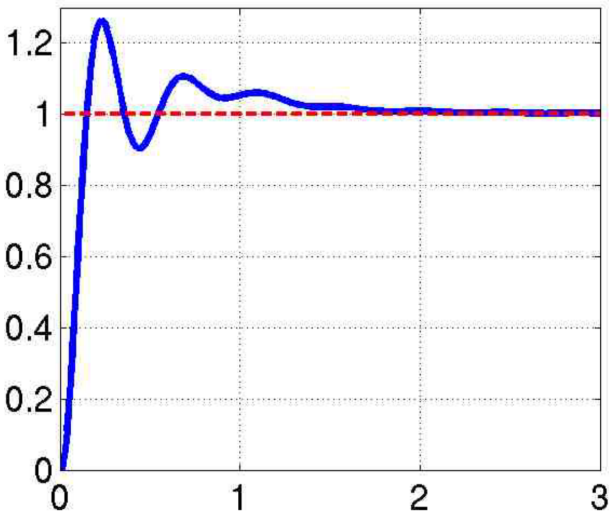

Figure 4: The response of a PI controller (Chen, 2011)

1.1.3 Derivative term

The derivative term $D(t)$ within a PID system is the derivative of the error over time. This means that the derivative term adds the change of error to the PID equation.

$$D(t) = \frac{de(t)}{dt}$$

The term cannot be used on its own since it only reacts to changes in the error caused by other terms. In practice the derivative term is used to dampen the other terms and remove heavy oscillations in the controller output.

The derivative term is however prone to noise in the input, because it reacts to changes in the error. Noise is the change in input that is irrelevant to the system, this can for example be a high frequency fluctuation as shown in the following graph.

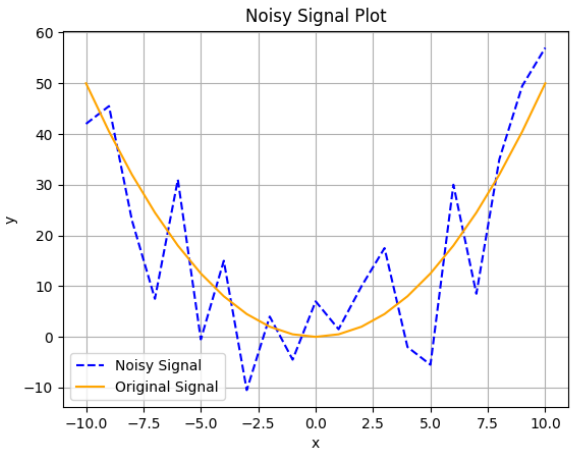

Figure 5: Graph showing noise in a signal (Cheyns, 2025a)

This vulnerability is why a noise filter is often applied when implementing a derivative term, this filter can smoothen out the noisy signal to make it usable for the derivative term. (x-engineer.org, n.d.)

To apply the derivative term to our thermostat example, the change in error from the previous error to the current error is calculated and divided by the sampling time to get the derivative term. Applying the derivative and proportional term, by combining them, to our thermostat example will result in a smaller change in power where the proportional term would output a fast change, allowing the system to negate possible overshoot.

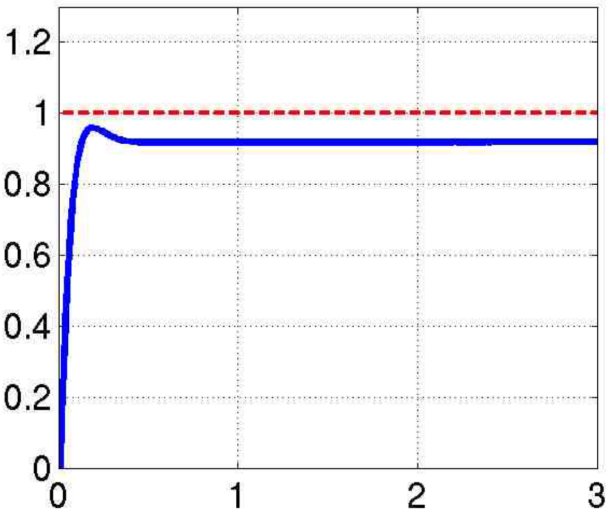

Figure 6: Graph showing the response of a PD controller (Chen, 2011)

Sufyan, 2025 Shoujun and Weiguo, 2011

1.1.4 Summary

The three terms within the PID system complement each other to achieve an optimal controller. Where one term comes short another term will take over on a tuned PID controller.

The proportional term reacts quickly to the error, the integral term ensures reaching the setpoint and the derivative term dampens possible overshoot and oscillations. Combining all factors will yield the following basic PID equation:

$$u(t) = K_P \cdot e(t) + K_I \cdot \int e(t)dt + K_D \cdot \frac{de(t)}{dt} \quad (1)$$

This equation is the most basic PID controller. In reality controllers will often have extra subsystems incorporated such as a filter for the derivative term or a form of anti-windup for the integral term. Controllers can also be combined with other logic to set a maximal output or change PID constants while running.

When applying previous full PID controller (1) to our thermostat example, the proportional term will quickly start heating the room, the integral term will ensure that the requested temperature is reached, and the derivative term will dampen possible overshoot caused by the integral term. This will result in a good working thermostat capable of quickly reaching and then maintaining the requested temperature. This all while minimizing overshoot and oscillations.

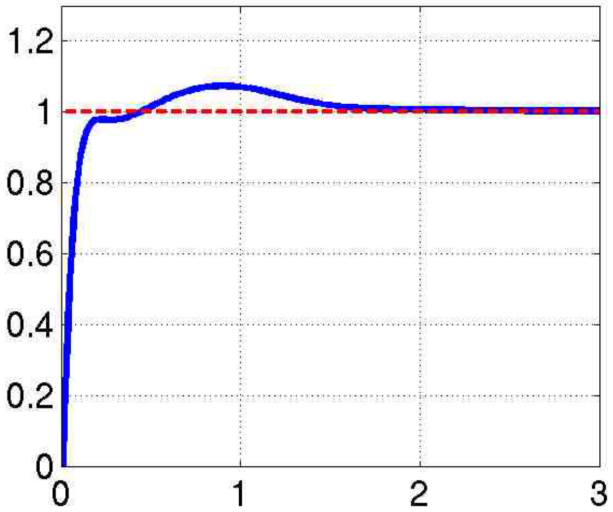

Figure 7: Graph showing the response of a full PID controller (Chen, 2011)

1.2 PID tuning

As discussed in previous subchapter, Proportional Integral Derivative (PID) controllers are useful tools when implemented and tuned right and can provide optimal machine control by quickly reaching a desired point and keeping the output stable. PID tuning is the act of adjusting the PID constants K_P, K_I and K_D to acquire a working controller for your desired plant. This can be done both manually or automatically using different methods.

1.2.1 Basic manual tuning concept

Tuning can be done using different methods, but each of them rely on the same basic concepts. Foremost, tuning heavily relies on trial

and error, repeating adjustments and analyzing the outcome to reach a tuned system. The three constant affect the controller in different ways.

Proportional constant

Increasing the proportional constant will make the PID controller reach the setpoint faster by increasing the initial speed. When the proportional constant is too high, the system becomes unstable and causes heavy fluctuations or overshoot.

Integral constant

Increasing integral action causes the controller to eliminate possible steady state error faster, but an integral constant set too high will cause very aggressive and wild fluctuations. This is especially the case if no form of anti-windup is implemented in the controller.

Differential constant

The differential constant sets the amount of damping that will be applied to the system, slowing the system down on large changes. A differential constant set too high can however cause the system to slow down or destabilize the system. The differential factor is even more prone to destabilization without a filter, filtering out possible noise.

(Sufyan, 2025) (Shoujun & Weiguo, 2011)

1.2.2 Ziegler-Nichols

The Ziegler-Nichols method was originally developed for analog PID like controllers by Ziegler and Nichols in 1942. (Ziegler & Nichols, 1942)

Later on, when digital PID controllers became common, the original method was adapted and is still widely used today.

The Ziegler-Nichols tuning method is a tuning method in which PID constants are determined based on the maximal value of P_K where the controller output starts to fluctuate. This value is called the ultimate gain K_U and can be found by setting the integral and differential constants to zero and adjusting the proportional constant until the output starts to fluctuate consistently.

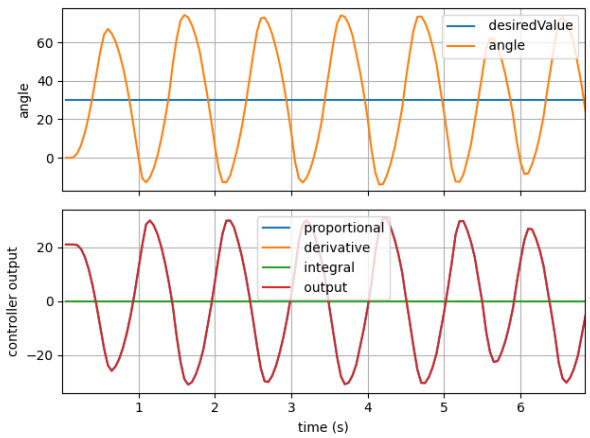

Figure 8: Graph showing a controller set to ultimate gain (Cheyns, 2025b)

When having found this value K_U, the period of the fluctuations is measured and called the ultimate period P_U. Using both the ulti-

mate gain and period, the PID constants can be determined using empirically determined functions. (Woolf, 2020)

Over time, many adaptations to the original Ziegler-Nichols functions were made to optimize the tuning method to different situations. The original Ziegler-Nichols functions were only made to tune P, PI or PID controllers, later on functions to tune PD controllers were also determined.

Controller	K_P	K_I	K_D
P	$0.5K_U$	-	-
PI	$0.45K_U$	$\frac{1.2K_U}{P_U}$	-
PD	$0.8H_U$	-	$0.1K_UP_u$
PID	$0.6K_U$	$\frac{1.2K_U}{P_U}$	$0.075K_UP_U$

Table 1: Adjusted Ziegler-Nichols tuning rules for different controller types. (Sedberg, 2025)

These constants can then be used as a starting point for further manual tuning. They will most likely not provide a fully tuned controller, and further adjustments will be necessary. (VanDoren, 1998)

2 RESEARCH METHOD

2.1 PID uses in robotics

PID controllers can be used in a lot of different plant types to reach precise values. Within robotics, precise autonomous movement is often required. Here as well, PID controllers are often implemented to achieve this.

A common use in competitive robotics such as VEX Robotics is to control a robot autonomously to turn a certain amount of degrees. To implement a PID controller for turning a bot, the sensor is taken from a gyroscope or using wheel encoders. As actuators, the motors that drive the wheels of the robot are used.

The PID controller will receive the current angle from the sensor, compare it to a desired angle and set the motor speeds accordingly. This allows the robot to turn precisely to the desired angle.

2.1.1 Python implementation within VEX robotics

A PID controller can be implemented into VEX using python code. The following code snippet contains a PID controller function.

```
1  def PID(desiredValue: int, tolerance:
   ↪  float, KP,KI,KD,yourSensor):
2      """Run PID loop until the sensor
       ↪  reaches desiredValue within
       ↪  tolerance.
3      This method updates output. It does
       ↪  not apply the output to
       ↪  actuators
4      """
5      previousError = 0
6      totalError = 0
7      i = 0
8      while abs(desiredValue -
       ↪  yourSensor()) > tolerance:
9          i += 1
10         error = desiredValue -
           ↪  yourSensor()
11         derivative = (error -
           ↪  previousError) / (50)
```

```
        totalError += error
        output = error * KP + derivative * KD + (totalError * 50) * KI
        wait(50)
        previousError = error
```

This controller can be further adapted to be used in VEX drivetrains by adding output to motors and calculating the proper angles like in turnPID (p. 14). To gather all PID-data used in this paper a similar program was used (turnPID p. 14), adding all PID terms onto a CSV file after each iteration for further analyzation.

Using a control loop that automatically saves PID terms, output and angle is useful because of its ease of analyzation. The gathered data can then be viewed using a CSV reader or plotted using a program such as plotter.py. (Cheyns, 2025c) This allows for viewing PID behavior precisely and after the initial test and therefore finetuning the PID controller.

In turnPID (p. 14) a check whether the PID controller has actually stabilized and not only reached the setpoint is implemented as well. This has been done by storing the previous errors onto a list and then checking whether they are all absolutely smaller than a tolerance around the setpoint.

The program also includes a speed cap, allowing the user to set a maximal turn speed in percentage. This is done by comparing the PID output with the maximal speed, if the controller outputs a speed higher than the cap, the cap speed gets used. To limit potential integral windup during maximal speed, the integral is also kept at 0 if the cap is reached.

turnPID can easily be reused as it is written as a python class. Implementation can be done by initializing and then calling needed functions in code.

An example of initializing turnPID:

```
rotatePID = turnPID(
    yourSensor= gyro.heading,
    brain = brain,
    leftMotorGroup=left,
    rightMotorGroup=right,
    speedCap= 20,
    KP = 0.34,
    KI = 0.13,
    KD = 0.014
    )
```

3 RESULTS

Using the code discussed in the previous chapter, I tuned our VEX robot, yielding multiple SCV files. They were then saved with the used tuning method, PID constants, setting and angles. This allowed for clear PID tuning examples graphed using plotter.py. (Cheyns, 2025c)

These generated and tuned graphs can then be compared with the ideal graphs used in the introduction.

To tune the full PID controller, the Ziegler-Nichols method was used and then manually tuned. To prevent integral windup a speedCap was also implemented at 20% motor speed.

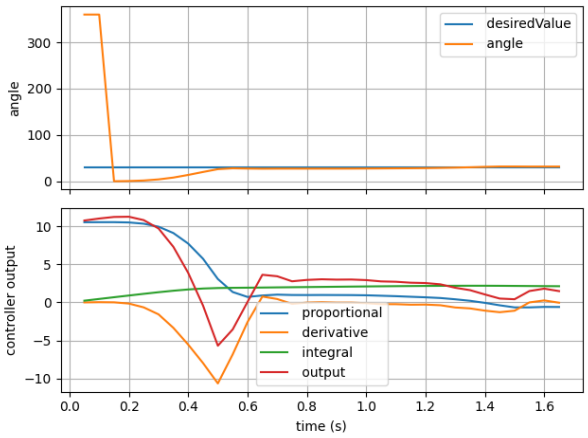

Figure 9: Graph showing a self-tuned PID controller

When comparing the generated controller (fig. 9) with the ideal (fig. 10) one, we can see that the self-made controller has a slower initial rise but cleanly reaches the setpoint and stabilizes like the ideal controller.

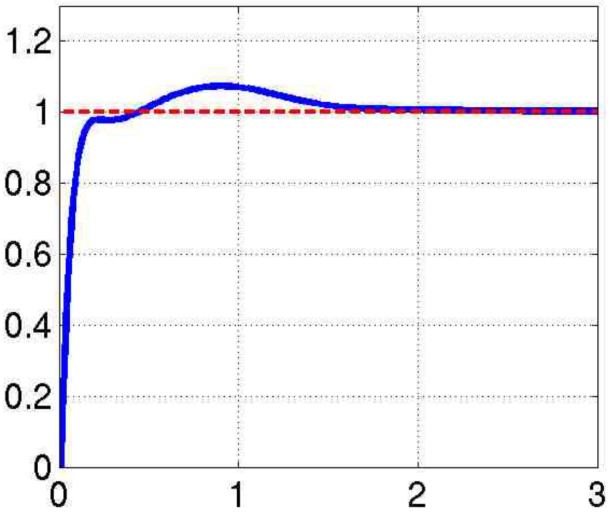

Figure 10: Graph showing an ideal PID controller

The following graph was made using our VEX robot with the PID constants: speedCap = 20, KP = 0.34, KI = 0.13, KD = 0.0.

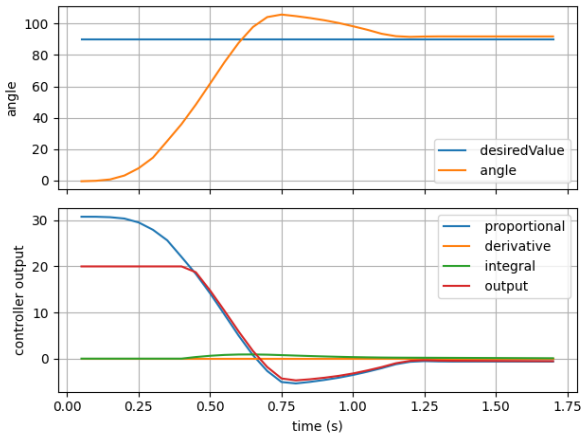

Figure 11: Graph showing a self-tuned PI controller

Just like the full PID controller, the self-made PI controller (fig. 11) has a smaller initial rise than the ideal controller (fig. 12). In this PI controller that was to be expected as a speedCap with integral clamp was implemented.

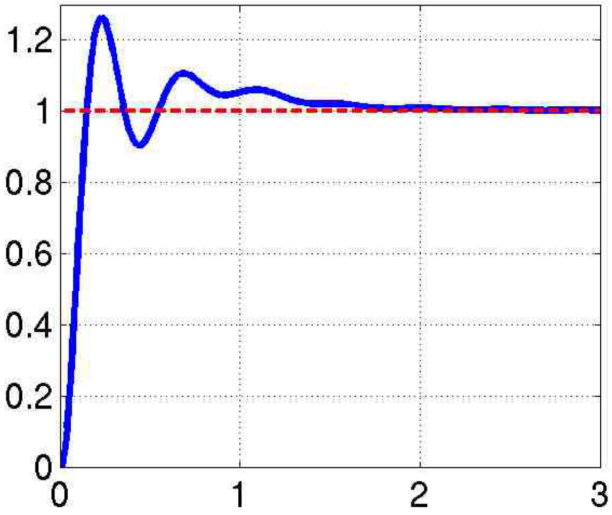

Figure 12: Graph showing an ideal PI controller

The next controller is a P only controller, generated using the Ziegler-Nichols method. It also has a speedCap set to 20% to diminish the initial overshoot.

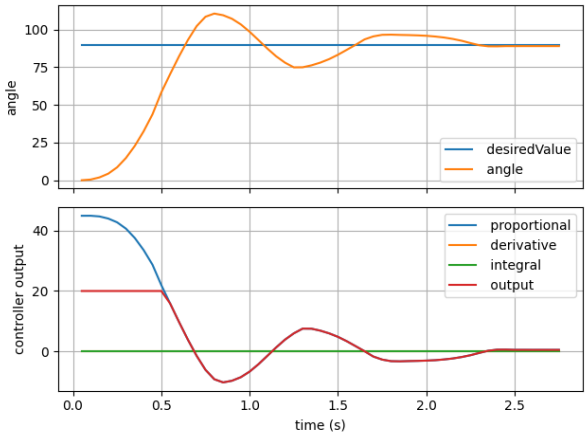

Figure 13: Graph showing a self-tuned P controller

When analyzing the P controllers, the ideal/example P controller (fig. 14) shows a steady state error while the self-tuned (fig. 13) one does not. This can be explained by the fact that different plants have different characteristics, so controllers behave slightly different.

The Vex robot didn't have a steady state error in this example, but when used on another surface it could acquire a steady state error. That is why an integral term was also implemented in the final PID controller used during competition, resulting in a more robust controller.

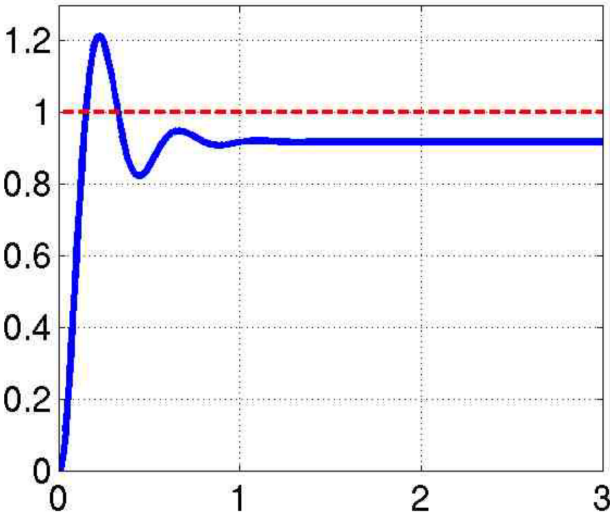

Figure 14: Graph showing an example P controller

The following graph shows a self-tuned PD controller made by slowly adding a differential term to the by Ziegler Nichols generated P controller.

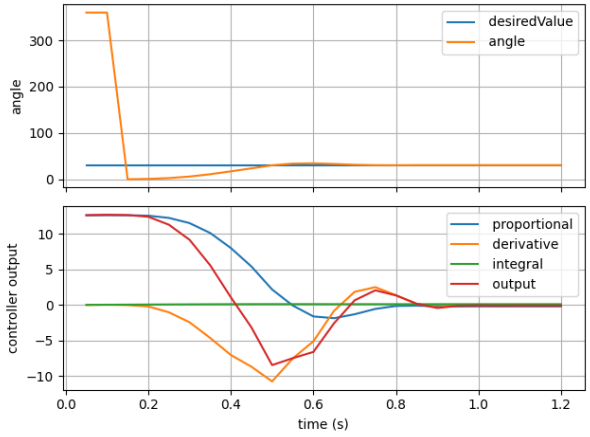

Figure 15: Graph showing a self-tuned PD controller

Just like the P controller, the PD example (fig. 6) shows a steady state error while the self-tuned (fig. 15) one does not. Other than that steady state error, the self-made one and the example show a common initial overshoot

before stabilizing.

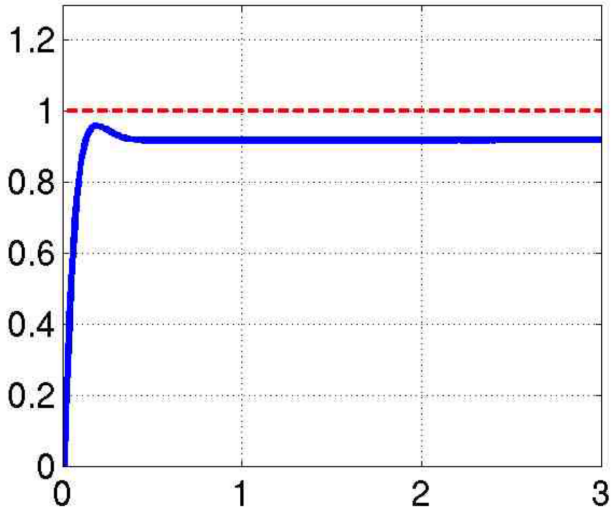

Figure 16: Graph showing an example PD controller

4 DISCUSSION

PID controllers are a valuable form of machine control to be implemented into competitive robotics such as VEX. They allow for precise movement and turns, making autonomous robot control more reliable.

4.1 VEX robotics

Within VEX robotics, robots must compete in two different categories: robot skills and competition.

In both these categories the bot must perform autonomous movement. In robot skills the bot must score as many points possible within a minute, this all autonomously. Within competition, robots get a fifteen-second autonomous time before manual control in which points can be scored.

To score these points, robots must traverse the field and interact with objects and field elements. This must all be done as precisely and quickly as possible to score the most points.

Controlling the motors can be done by turning them on for a limited time, or by using sensor inputs. Of those two, sensor inputs are much more reliable as they take slight variations in the field and motors in account.

A simple system would be to just turn motors on until they reach a certain sensor readout. However, this would be inefficient as the system would over or under-reach the setpoint.

Hence, another form of machine control in which the error gets used is required. PID systems offer all that is needed. They use the error to calculate motor speed and are able to precisely control the bot on different field situations. Furthermore, multiple changes such as implementing a filter or integral clamp can be made to a PID controller to set it up as desired.

5 CONCLUSION

The paper explained the basic workings of a PID controller, this by explaining all PID terms and mathematically analyzing the structure of a PID controller. It also contains a guide and research on tuning a PID

controller so that the reader can properly tune their own controllers. Furthermore, it explained the structure of a PID controller made in python.

A python program for use in VEX robotics where PID output is mapped to a CSV file to aid in PID tuning was also introduced in the paper.

The reader should now be able to create their own PID controller or use the provided code to control their VEX robotics bot during autonomous control. Using the provided tuning information the reader also has the ability to tune their PID controller manually or by using the Ziegler-Nichols method.

Acknowledgements

This paper was not possible without the support of my friends and robotics team members. The support and help from my thesis teachers Van Nuffel Joke and Vanhaecht Lies allowed me to start this paper along with the continuing support of our robotics coach Vlummens An. Of course, I also thank my parents for supporting me throughout my life.

References

Chen, E. Y. (2011). Controlsandsignals-mit course. https://web.mit.edu/6.186/2011/Lectures/controls/ControlsandSignals.pdf

Sedberg, L. (2025). *Simple pid tuning generator (beta).* https://lucassedberg.com/tools/control/pid

Shoujun, S., & Weiguo, L. (2011). Application of improved pid controller in motor drive system. In T. Mansour (Ed.), *Pid control, implementation and tuning.* INTECH d.o.o.

Sufyan, M. (2025). *Pid explained: Theory, tuning, and implementation of pid controllers.* https://www.wevolver.com/article/pid-explained-theory-tuning-and-implementation-of-pid-controllers

VanDoren, V. J. (1998). *Ziegler-nichols tuning methods.* https://www.controleng.com/ziegler-nichols-tuning-methods/

VanDoren, V. J. (2003). *Pid still the one.* https://www.controleng.com/pid-still-the-one/

Woolf, M. (2020). *9.3: Pid tuning via classical methods.* https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Chemical_Process_Dynamics_and_Controls_(Woolf)/09%3A_Proportional-Integral-Derivative_(PID)_Control/9.03%3A_PID_Tuning_via_Classical_Methods

x-engineer.org. (n.d.). *What is a low pass filter used for?* https://x-engineer.org/low-pass-filter/

Ziegler, J. G., & Nichols, N. B. (1942). Optimum settings for automatic controllers.

List of code

Cheyns, R. (2025a). noiseExample.py. https://github.com/Ruben-Cheyns/PID/blob/main/src/noiseExample.py

Cheyns, R. (2025b). PID.py. https://github.com/Ruben-Cheyns/PID

Cheyns, R. (2025c). plotter.py. https://github.com/Ruben-Cheyns/PID/blob/main/src/plotter.py

Code appendix

Listing 1: turnPID Code (Cheyns, 2025b)

```
class turnPID(PID):
    """PID controller specialized for turning a drivetrain (left/right motor groups).

    It computes a rotational output and sets velocities on left and right
    ↪  MotorGroups.

    Constructor parameters:
        yourSensor: callable returning current heading/angle
        brain: Brain instance
        leftMotorGroup, rightMotorGroup: MotorGroup instances to apply rotation
        speedCap: a maximal value at wich the controller stops working and outputs
        ↪  this fixed value
        KP, KI, KD: PID gains
    """

    def __init__(self, yourSensor, brain: Brain, leftMotorGroup: MotorGroup,
    ↪  rightMotorGroup: MotorGroup, speedCap: int = 100, KP: float = 1, KI: float =
    ↪  0, KD: float = 0):
        self.KP = KP
        self.KI = KI
        self.KD = KD
        self.left = leftMotorGroup
        self.right = rightMotorGroup
        self.yourSensor = yourSensor
        self.brain = brain
        self.output:float = 0
        self.speedCap:int = speedCap

    def run (self, desiredValue: int, tolerance: float, settleTime: float = 0.5):
        """Run turn PID and set motor velocities until target heading stabilized.
        desiredValue: the angle you wish to turn to in degrees
        tolerance: the tolerance used to detect setpoint stabilization in degrees
        settleTime: how long the setpoint must be stable to exit the pid loop in
        ↪  seconds
        """
        # starts up drivetrain motors with speed set to zero
        self.right.spin(FORWARD, 0)
        self.left.spin(FORWARD, 0)

        # necessary local variables, totalError over the entire loop and i:
        ↪  iterations
        totalError:float = 0.0
        i = 0

        # calculates the initial error, paying mind to the smallest angles.
        if desiredValue - self.yourSensor() > 0:
            if desiredValue - self.yourSensor() <= 180:
                error:float = desiredValue - self.yourSensor()
            elif desiredValue - self.yourSensor() > 180:
```

```
            error:float = desiredValue - self.yourSensor() - 360
        else:
            if desiredValue - self.yourSensor() >= -180:
                error:float = desiredValue - self.yourSensor()
            elif desiredValue - self.yourSensor() < -180:
                error:float = 360 + (desiredValue - self.yourSensor())

        # adds initial error to the errorList used in detecting stabilization and previousError used in the derivative
        errorList = [error]
        previousError:float = error

        # runs the PID loop until setpoint is stabilized by detecting the absolute smallest error
        while abs(max(errorList, key=abs)) > tolerance:
            # counts iterations
            i += 1
            #calculates error, paying mind to smallest angles
            if desiredValue - self.yourSensor() > 0:
                if desiredValue - self.yourSensor() <= 180:
                    error:float = desiredValue - self.yourSensor()
                elif desiredValue - self.yourSensor() > 180:
                    error:float = desiredValue - self.yourSensor() - 360
            else:
                if desiredValue - self.yourSensor() >= -180:
                    error:float = desiredValue - self.yourSensor()
                elif desiredValue - self.yourSensor() < -180:
                    error:float = 360 + (desiredValue - self.yourSensor())

            # calculates derivative using error, previousError and sampling time
            derivative = (error - previousError) / 0.050
            # adds current error to totalError used in integral term
            totalError += error
            # sets output to the total PID equation or the speedCap
            self.output = min(error * self.KP + derivative * self.KD + (totalError * 0.050) * self.KI, (self.speedCap if error * self.KP + derivative * self.KD + (totalError * 0.050) * self.KI > 0 else -self.speedCap), key=abs)
            # sets totalError to zero if speedCap reached to prevent integral windup
            if abs(self.output) == self.speedCap:
                totalError = 0
            # sets motor velocity to the PID output
            self.left.set_velocity(self.output, PERCENT)
            self.right.set_velocity(-self.output, PERCENT)
            # waits 50 MS to lighten program load
            wait(50)
            # sets previousError to current error for derivative term
            previousError = error
            # adds error to errorList and deletes the first error if the list is longer than the settleTime divided by sampling time
            errorList.append(error)
            if len(errorList) > settleTime/0.050:
```

```
90                  errorList.pop(0)
91
92      def graph(self, desiredValue: int, tolerance: float, settleTime: float = 0.5,
        ↪  sd_file_name = "pidData.csv", stopButton = False):
93          """Runs loop similar to PID.run but saves a CSV containing PID data.
94
95          CSV columns:
96              time, proportional, derivative, integral, output, desiredValue, angle
97          """
98          # makes a stopButton if set to true to manually stop the loop
99          if stopButton:
100             stop = button(60, 220, 250, 10, Color.RED, "terminate")
101             stop.draw()
102             brain.screen.render()
103
104         # csv variables to store PID data
105         csvHeaderText:str = "time, proportional, derivative, integral, output,
            ↪  desiredValue, angle"
106         data_buffer:str = csvHeaderText + "\n"
107
108         # starts up drivetrain motors with speed set to zero
109         self.right.spin(FORWARD, 0)
110         self.left.spin(FORWARD, 0)
111
112         # necessary local variables, totalError over the entire loop and i:
            ↪  iterations
113         totalError:float = 0.0
114         i = 0
115
116         # calculates the initial error, paying mind to the smallest angles.
117         if desiredValue - self.yourSensor() > 0:
118             if desiredValue - self.yourSensor() <= 180:
119                 error:float = desiredValue - self.yourSensor()
120             elif desiredValue - self.yourSensor() > 180:
121                 error:float = desiredValue - self.yourSensor() - 360
122         else:
123             if desiredValue - self.yourSensor() >= -180:
124                 error:float = desiredValue - self.yourSensor()
125             elif desiredValue - self.yourSensor() < -180:
126                 error:float = 360 + (desiredValue - self.yourSensor())
127
128         # adds initial error to the errorList used in detecting stabilization and
            ↪  previousError used in the derivative
129         errorList = [error]
130         previousError:float = error
131
132         # runs the PID loop until setpoint is stabilized by detecting the absolute
            ↪  smallest error
133         while abs(max(errorList, key=abs)) > tolerance:
134             # counts iterations
135             i += 1
136             #calculates error, paying mind to smallest angles
```

```
137             if desiredValue - self.yourSensor() > 0:
138                 if desiredValue - self.yourSensor() <= 180:
139                     error:float = desiredValue - self.yourSensor()
140                 elif desiredValue - self.yourSensor() > 180:
141                     error:float = desiredValue - self.yourSensor() - 360
142             else:
143                 if desiredValue - self.yourSensor() >= -180:
144                     error:float = desiredValue - self.yourSensor()
145                 elif desiredValue - self.yourSensor() < -180:
146                     error:float = 360 + (desiredValue - self.yourSensor())
147 
148             # calculates derivative using error, previousError and sampling time
149             derivative = (error - previousError) / 0.050
150             # adds current error to totalError used in integral term
151             totalError += error
152             # sets output to the total PID equation or the speedCap
153             self.output = min(error * self.KP + derivative * self.KD + (totalError *
                ↪  0.050) * self.KI, (self.speedCap if error * self.KP + derivative *
                ↪  self.KD + (totalError * 0.050) * self.KI > 0 else -self.speedCap),
                ↪  key=abs)
154             # sets totalError to zero if speedCap reached to prevent integral windup
155             if abs(self.output) == self.speedCap:
156                 totalError = 0
157             # sets motor velocity to the PID output
158             self.left.set_velocity(self.output, PERCENT)
159             self.right.set_velocity(-self.output, PERCENT)
160             # waits 50 MS to lighten program load
161             wait(50)
162             # sets previousError to current error for derivative term
163             previousError = error
164             # adds error to errorList and deletes the first error if the list is
                ↪  longer than the settleTime divided by sampling time
165             errorList.append(error)
166             if len(errorList) > settleTime/0.050:
167                 errorList.pop(0)
168 
169             # save one row of PID data to buffer
170             data_buffer += str(i * 0.050) + ","
171             data_buffer += "%.3f" % (error*self.KP) + ","
172             data_buffer += "%.3f" % (derivative * self.KD)  + ","
173             data_buffer += "%.3f" % (totalError * 0.050 * self.KI) + ","
174             data_buffer += "%.3f" % self.output + ","
175             data_buffer += str(desiredValue) + ","
176             data_buffer += "%.3f" % self.yourSensor() + "\n"
177 
178             # breaks the pid loop if the stopButton is pressed
179             if stopButton and
                ↪  stop.isPressed(self.brain.screen.x_position(),self.brain.screen.y_position()):
180                 break
181 
182         # saves the data buffer onto the SD card as a SCV
183         self.brain.sdcard.savefile(sd_file_name, bytearray(data_buffer, 'utf-8'))
```